Universidad de los Llanos · Ingeniería de Sistemas · Bases de Datos 2026-II

Informe de la base de datos
Proyecto #13: Sistema de Ventas por Internet

Juan Esteban Ramos Triviño 160005465
Andrés Jerónimo Puerta López 160005490
David Santiago Díaz Riveros 160005319

Docente: Jesús Reyes Carvajal · 8 de noviembre de 2026

Introducción

La base de datos guarda la operación de una tienda que vende artículos por Internet: clientes y sus cuentas, catálogo con fotos, pedidos, artículos de cada pedido y pagos. Está en PostgreSQL 16, publicada en Railway, y la usa la aplicación web en PHP del mismo proyecto.

El objetivo es que las reglas del negocio las haga cumplir la base de datos y no el programador. Por ejemplo: no dejar el inventario en negativo, no aceptar precios o montos negativos, no borrar un cliente que ya compró y no dejar que un cliente vea las compras de otro. Así, aunque la aplicación tenga un error, los datos se mantienen consistentes y protegidos.

La base se crea con tres scripts, en este orden:

Orden	Script	Qué hace
1	`ventas_internet.sql`	Crea la base `proyecto`, las 7 tablas principales con sus restricciones, los 2 disparadores y los datos de prueba.
2	`login_roles.sql`	Agrega las cuentas (contraseña y rol en `cliente`), crea los roles `rol_visitante`, `rol_cliente` y `rol_admin` con sus permisos y activa la seguridad por filas (RLS).
3	`migracion_imagenes_mis_datos.sql`	Crea la tabla `articulo_imagen` para las fotos y la función `fn_actualizar_mis_datos`.

Los scripts 2 y 3 se pueden ejecutar varias veces sin borrar datos (son idempotentes). Además, al encender la aplicación en Railway, `scripts/crear_admin.php` crea o actualiza la cuenta del administrador con los datos de las variables de entorno, para que su contraseña nunca quede escrita en el repositorio.

1. Objetivos

Objetivo general

Desarrollar un sistema web de ventas por Internet, con una base de datos en PostgreSQL y una aplicación en PHP sin frameworks sobre Linux, que permita registrar, actualizar y consultar los artículos, clientes, ciudades, ventas y pagos de una tienda, y hacer el seguimiento de cada venta desde el pedido hasta la entrega.

Objetivos específicos

- Diseñar el modelo entidad-relación y el modelo relacional de la tienda, normalizados hasta la tercera forma normal (3FN).
- Crear la base de datos con llaves primarias y foráneas, restricciones `CHECK` para que precios, cantidades, inventario y montos no sean negativos, y disparadores que controlen el inventario y las fechas de pago.
- Construir un panel de administración para gestionar artículos, categorías, clientes, ciudades, ventas y pagos, y una tienda donde los clientes registrados compran con un carrito y corrigen sus propios datos.
- Hacer el seguimiento de cada venta por sus estados (Pendiente, Pagada, Enviada, Entregada o Cancelada) y registrar sus pagos por PSE, tarjeta Visa o Mastercard, Efecty o transferencia bancaria.
- Validar los datos de entrada en la aplicación y en la base, y proteger la información con cuentas, roles y seguridad por filas, para que cada cliente vea solo sus compras.
- Publicar el sistema en Railway para usarlo desde cualquier navegador.

2. Justificación

Una tienda que vende por Internet maneja cada día pedidos, inventario y pagos de clientes de distintas ciudades. Si esa información se lleva a mano o en hojas de cálculo, aparecen errores comunes: vender un artículo que ya se agotó, perder el rastro de un pedido, registrar un pago con una fecha imposible o no saber cuánto le falta pagar a un cliente.

Este sistema guarda todo en una base de datos relacional que hace cumplir las reglas del negocio por sí misma: el inventario se descuenta solo al vender y nunca queda negativo, un pago no puede tener fecha anterior a su venta, no se puede borrar un cliente o un artículo que ya tiene ventas, y el total de cada venta se calcula siempre a partir de sus artículos, así que nunca queda desactualizado.

Además, el cliente compra desde la tienda y consulta el estado de sus pedidos sin intermediarios, y el administrador ve en un solo lugar qué ventas faltan por pagar, enviar o entregar. Como usa solo herramientas libres (PostgreSQL, PHP y Linux) y está publicado en la web, se puede usar desde cualquier lugar sin pagar licencias.

3. Modelo entidad-relación

El modelo tiene 7 entidades y 6 relaciones. Dentro de cada entidad están sus atributos, con la llave primaria subrayada. Imagen es una entidad débil: no tiene llave propia y solo existe si existe su artículo.

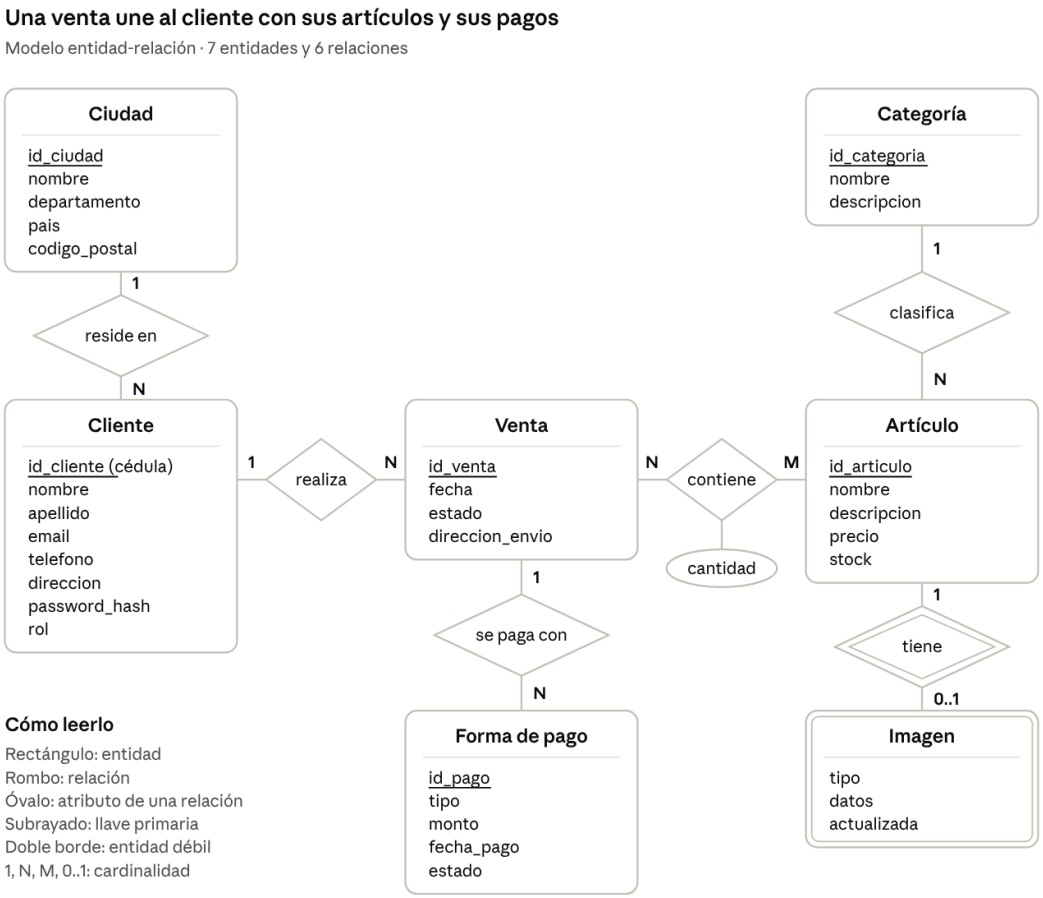


Figura 1. Modelo entidad-relación

Relación	Entidades	Cardinalidad	Qué significa
reside en	Cliente – Ciudad	N:1	Cada cliente vive en una ciudad; en una ciudad viven muchos clientes.
realiza	Cliente – Venta	1:N	Un cliente hace muchas compras; cada venta es de un solo cliente.

Relación	Entidades	Cardinalidad	Qué significa
clasifica	Categoría – Artículo	1:N	Una categoría agrupa muchos artículos; cada artículo está en una sola categoría.
contiene	Venta – Artículo	N:M	Una venta lleva varios artículos y un artículo aparece en muchas ventas. La relación tiene su propio atributo, `cantidad`.
se paga con	Venta – Forma de pago	1:N	Una venta puede recibir varios pagos (por ejemplo, un abono y el resto); cada pago es de una sola venta.
tiene	Artículo – Imagen	1:0..1	Un artículo tiene como máximo una foto, y la foto pertenece a un solo artículo.

4. Modelo relacional

La base tiene 8 tablas unidas por 7 llaves foráneas. Salen de las 6 entidades del modelo entidad-relación, más la tabla puente `detalle_venta`, que resuelve la relación muchos a muchos entre ventas y artículos, y la tabla `articulo_imagen`, con la foto de cada artículo.

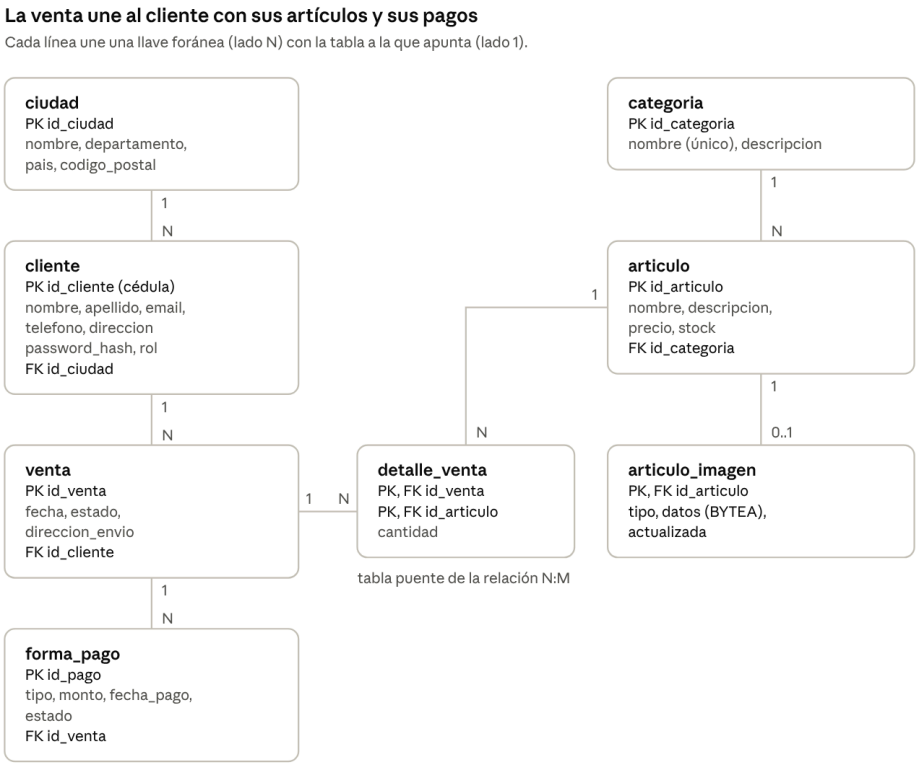


Figura 2. Modelo relacional

`venta` está en el centro: cada venta pertenece a un cliente, tiene una o más líneas de detalle (cada una con un artículo) y recibe uno o más pagos. Cada artículo tiene cero o una foto (relación 1 a 0..1), y `cliente` guarda también los datos de la cuenta: `password_hash` y `rol`.

Así se pasó del modelo entidad-relación a las tablas:

- Cada entidad es una tabla, con la misma llave primaria.
- Las cuatro relaciones 1:N se vuelven una llave foránea en la tabla del lado N: `cliente.id_ciudad`, `venta.id_cliente`, `articulo.id_categoria` y `forma_pago.id_venta`.
- La relación N:M *contiene* se vuelve la tabla `detalle_venta`, con llave primaria compuesta (`id_venta`, `id_articulo`) y el atributo `cantidad`.
- La entidad débil Imagen se vuelve `articulo_imagen`. Su llave primaria `id_articulo` es también llave foránea hacia `articulo`, y por eso cada artículo tiene como máximo una foto.

5. Diccionario de datos

Para cada tabla: el nombre de cada campo, su tipo en PostgreSQL, si acepta valores nulos, si es llave y qué guarda. Las reglas de cada campo se explican con más detalle en la sección 6.

ciudad

Campo	Tipo	Nulo	Llave	Descripción
id_ciudad	SERIAL	No	PK	Número de la ciudad; lo asigna la base.
nombre	VARCHAR(80)	No	—	Nombre de la ciudad.
departamento	VARCHAR(80)	No	—	Departamento al que pertenece.
pais	VARCHAR(60)	No	—	País; por defecto 'Colombia'.
codigo_postal	VARCHAR(15)	No	—	Código postal.

cliente

Campo	Tipo	Nulo	Llave	Descripción
id_cliente	INT4	No	PK	Cédula del cliente; mayor que 0.
nombre	VARCHAR(80)	No	—	Nombres.
apellido	VARCHAR(80)	No	—	Apellidos.
email	VARCHAR(120)	No	UNIQUE	Correo; es también el usuario para iniciar sesión.
telefono	VARCHAR(20)	No	—	Teléfono de contacto.
direccion	VARCHAR(150)	No	—	Dirección de entrega por defecto.
id_ciudad	INT4	No	FK → ciudad	Ciudad donde vive.
password_hash	TEXT	Sí	—	Contraseña cifrada; obligatoria para el administrador.
rol	VARCHAR(20)	No	—	'admin' o 'cliente'; por defecto 'cliente'.

categoria

Campo	Tipo	Nulo	Llave	Descripción
id_categoria	SERIAL	No	PK	Número de la categoría; lo asigna la base.
nombre	VARCHAR(80)	No	UNIQUE	Nombre de la categoría, sin repetir.
descripcion	VARCHAR(255)	Sí	—	Descripción opcional.

articulo

Campo	Tipo	Nulo	Llave	Descripción
id_articulo	SERIAL	No	PK	Número del artículo; lo asigna la base.
nombre	VARCHAR(120)	No	—	Nombre del artículo.
descripcion	VARCHAR(255)	Sí	—	Descripción opcional.
precio	NUMERIC(12,2)	No	—	Precio de venta en pesos; mayor que 0.
stock	INT4	No	—	Unidades disponibles; 0 o más, por defecto 0. Lo descuenta el disparador al vender.
id_categoria	INT4	No	FK → categoria	Categoría del artículo.

articulo_imagen

Campo	Tipo	Nulo	Llave	Descripción
id_articulo	INT4	No	PK, FK → articulo	Artículo dueño de la foto.
tipo	VARCHAR(20)	No	—	Formato: image/jpeg, image/png o image/webp.
datos	BYTEA	No	—	Bytes de la imagen, de 1 byte a 2 MB.
actualizada	TIMESTAMP	No	—	Fecha y hora en que se subió; por defecto, el momento actual.

venta

Campo	Tipo	Nulo	Llave	Descripción
id_venta	SERIAL	No	PK	Número de la venta o pedido.
fecha	DATE	No	—	Fecha del pedido; por defecto, hoy.
estado	VARCHAR(30)	No	—	Pendiente, Pagada, Enviada, Entregada o Cancelada; por defecto Pendiente.
direccion_envio	VARCHAR(150)	No	—	Dirección a la que se envía el pedido.
id_cliente	INT4	No	FK → cliente	Cliente que compró.

detalle_venta

Campo	Tipo	Nulo	Llave	Descripción
id_venta	INT4	No	PK, FK → venta	Venta a la que pertenece la línea.
id_articulo	INT4	No	PK, FK → articulo	Artículo vendido.
cantidad	INT4	No	—	Unidades vendidas; mayor que 0.

forma_pago

Campo	Tipo	Nulo	Llave	Descripción
id_pago	SERIAL	No	PK	Número del pago; lo asigna la base.
tipo	VARCHAR(40)	No	—	PSE, Tarjeta Visa, Tarjeta Mastercard, Efecty o Transferencia Bancaria.
monto	NUMERIC(12,2)	No	—	Valor pagado; mayor que 0.
fecha_pago	DATE	No	—	Fecha del pago; por defecto, hoy. No puede ser anterior a la fecha de la venta.
estado	VARCHAR(30)	No	—	Pendiente, Aprobado o Rechazado; por defecto Aprobado.
id_venta	INT4	No	FK → venta	Venta que se paga.

6. Restricciones de cada tabla

Cada tabla tiene sus restricciones (*constraints*). PostgreSQL las revisa en cada `INSERT`, `UPDATE` y `DELETE`, y si un dato no las cumple, la operación se rechaza con un error. Hay seis tipos:

- **PK** (`PRIMARY KEY`): identifica cada fila; no se repite ni puede ser nula.
- **FK** (`FOREIGN KEY`): el valor tiene que existir en la tabla padre. `ON DELETE RESTRICT` impide borrar el padre si tiene hijos; `ON DELETE CASCADE` borra los hijos con él. Las 7 FK usan `ON UPDATE CASCADE`: si cambia el id del padre, el cambio pasa a los hijos.
- **CHECK**: una condición que cada fila debe cumplir.
- **UNIQUE**: el valor no se puede repetir en la columna.
- **NOT NULL**: la columna es obligatoria. Todas lo son, excepto las dos `descripcion` y `password_hash`.
- **DEFAULT**: el valor que se pone cuando no se escribe ninguno.

ciudad — ciudades donde viven los clientes

Restricción	Tipo	Regla	Por qué
pk_ciudad	PK	`id_ciudad` SERIAL	Identifica cada ciudad con un número automático
—	DEFAULT	`pais = 'Colombia'`	Casi todas las ciudades son de Colombia, así no hay que escribirlo

cliente — compradores registrados y sus cuentas

Restricción	Tipo	Regla	Por qué
pk_cliente	PK	`id_cliente` (cédula)	La cédula ya es un dato único y real
ck_cliente_id	CHECK	`id_cliente > 0`	Una cédula no puede ser 0 ni negativa
uq_cliente_email	UNIQUE	`email`	Dos clientes no pueden usar el mismo correo, que es con el que inician sesión

Restricción	Tipo	Regla	Por qué
fk_cliente_ciudad	FK → ciudad	RESTRICT / CASCADE	La ciudad tiene que existir, y no se puede borrar una ciudad con clientes
ck_cliente_rol	CHECK	`rol` es `admin` o `cliente`	Solo existen esos dos tipos de cuenta
ck_cliente_admin_con_clave	CHECK	`rol <> 'admin' OR password_hash IS NOT NULL`	Una cuenta de administrador siempre tiene contraseña
—	DEFAULT	`rol = 'cliente'`	Quien se registra en la tienda queda como cliente

Las columnas de la cuenta son `password_hash` (la contraseña cifrada con bcrypt; puede ser nula en los clientes que crea el administrador, que no pueden iniciar sesión) y `rol`. La contraseña nunca se guarda tal cual: solo su hash, que no se puede descifrar.

categoria — líneas del catálogo

Restricción	Tipo	Regla	Por qué
pk_categoria	PK	`id_categoria` SERIAL	Identifica cada categoría
uq_categoria_nombre	UNIQUE	`nombre`	Evita tener la misma categoría dos veces

articulo — productos a la venta

Restricción	Tipo	Regla	Por qué
pk_articulo	PK	`id_articulo` SERIAL	Identifica cada producto
ck_articulo_precio	CHECK	`precio > 0`	Nada se vende gratis ni a precio negativo
ck_articulo_stock	CHECK	`stock >= 0`	No se pueden vender unidades que no hay
—	DEFAULT	`stock = 0`	Un artículo nuevo empieza sin existencias
fk_articulo_categoria	FK → categoria	RESTRICT / CASCADE	Todo artículo tiene categoría, y no se puede borrar una categoría con artículos

El precio usa `NUMERIC(12,2)` y no `FLOAT`, para que el dinero se guarde exacto, sin errores de redondeo.

articulo_imagen — foto de cada artículo

Restricción	Tipo	Regla	Por qué
pk_articulo_imagen	PK	`id_articulo`	Cada artículo tiene como máximo una foto
fk_articulo_imagen_articulo	FK → articulo	CASCADE / CASCADE	La foto pertenece a un artículo que existe; si se borra el artículo, su foto se borra con él
ck_articulo_imagen_tipo	CHECK	`tipo` es `image/jpeg`, `image/png` o `image/webp`	Solo se aceptan formatos de imagen que muestran todos los navegadores
ck_articulo_imagen_tamano	CHECK	`octet_length(datos)` entre 1 y 2.097.152 bytes	Ninguna foto pesa más de 2 MB ni está vacía
—	DEFAULT	`actualizada = CURRENT_TIMESTAMP`	Guarda cuándo se subió la foto; la página la usa para que el navegador cargue la nueva al cambiarla

La foto se guarda dentro de la base, en la columna `datos` de tipo `BYTEA` (bytes), y no en una carpeta del servidor, porque Railway borra los archivos subidos en cada despliegue.

venta — encabezado de cada pedido

Restricción	Tipo	Regla	Por qué
pk_venta	PK	`id_venta` SERIAL	Es el número de la orden
ck_venta_estado	CHECK	Pendiente, Pagada, Enviada, Entregada o Cancelada	Solo se aceptan los estados reales de un pedido
—	DEFAULT	`estado = 'Pendiente'`, `fecha = CURRENT_DATE`	Todo pedido nace pendiente y con la fecha del día

Restricción	Tipo	Regla	Por qué
fk_venta_cliente	FK → cliente	RESTRICT / CASCADE	Toda venta tiene un dueño, y no se puede borrar un cliente que ya compró

detalle_venta — artículos de cada pedido

Restricción	Tipo	Regla	Por qué
pk_detalle_venta	PK compuesta	(`id_venta`, `id_articulo`)	Un artículo aparece una sola vez por venta; si se compran más, sube la cantidad
ck_detalle_cantidad	CHECK	`cantidad > 0`	Cada línea vende al menos 1 unidad
fk_detalle_venta	FK → venta	CASCADE / CASCADE	Si se borra la venta, sus líneas se borran con ella
fk_detalle_articulo	FK → articulo	RESTRICT / CASCADE	No se puede borrar un artículo que ya se vendió

forma_pago — pagos de cada pedido

Restricción	Tipo	Regla	Por qué
pk_forma_pago	PK	`id_pago` SERIAL	Identifica cada transacción
ck_forma_pago_tipo	CHECK	PSE, Tarjeta Visa, Tarjeta Mastercard, Efecty o Transferencia Bancaria	Solo se aceptan los medios de pago habilitados
ck_forma_pago_monto	CHECK	`monto > 0`	Un pago siempre tiene un valor positivo
ck_forma_pago_estado	CHECK	Pendiente, Aprobado o Rechazado	Son los estados posibles de una transacción
—	DEFAULT	`estado = 'Aprobado'`, `fecha_pago = CURRENT_DATE`	El caso normal es un pago aprobado el mismo día
fk_forma_pago_venta	FK → venta	RESTRICT / CASCADE	Todo pago pertenece a una venta, y no se puede borrar una venta con pagos

7. Secuencias (`_seq`)

Cada columna SERIAL crea una secuencia: un contador que asigna el siguiente id al insertar. Hay 5 (`ciudad_id_ciudad_seq`, `categoria_id_categoria_seq`, `articulo_id_articulo_seq`, `venta_id_venta_seq` y `forma_pago_id_pago_seq`). `cliente` no tiene secuencia porque usa la cédula, y `articulo_imagen` tampoco, porque su llave es el `id_articulo` del artículo al que pertenece. Una secuencia nunca retrocede, así que si un `INSERT` falla quedan huecos en la numeración, y eso es normal.

8. Disparadores y funciones

El proyecto exige al menos un disparador y una restricción de fechas; la base tiene dos disparadores que cubren ambas cosas. Un disparador es una función PL/pgSQL que PostgreSQL ejecuta sola cuando ocurre algo en una tabla. Los dos corren `BEFORE` (antes de guardar la fila), así que pueden cancelarla con `RAISE EXCEPTION`.

Disparador	Cuándo se ejecuta	Qué hace	Por qué
trg_descontar_stock_venta (función `fn_descontar_stock_venta`)	Antes de cada `INSERT` en `detalle_venta`	Lee y bloquea el stock del artículo (`FOR UPDATE`). Si no alcanza, aborta con "Stock insuficiente"; si alcanza, hace `stock = stock - cantidad`	El inventario se actualiza en cada venta sin depender de la aplicación, y dos compras simultáneas no venden la misma unidad
trg_validar_fecha_pago (función `fn_validar_fecha_pago`)	Antes de cada `INSERT` o `UPDATE` en `forma_pago`	Rechaza el pago si `fecha_pago` es anterior a la fecha de su venta	Es la restricción de fecha inicial / fecha final. Tiene que ser disparador porque un `CHECK` solo ve su propia fila y esta regla compara dos tablas

La función `fn_descontar_stock_venta` es `SECURITY DEFINER`, es decir, corre con los permisos de su dueño. Así, cuando un cliente compra desde la tienda, el disparador puede descontar el stock aunque `rol_cliente` no tenga permiso de `UPDATE` sobre `articulo`.

Además hay una función que no es disparador:

Función	Quién la usa	Qué hace	Por qué
`fn_actualizar_mis_datos(nombre, apellido, email, telefono, direccion, id_ciudad)`	La página Mis datos, con `rol_cliente`	Actualiza esos 6 datos solo en la fila del cliente con sesión iniciada (la cédula guardada en `app.id_cliente`). Si no hay sesión, aborta con `RAISE EXCEPTION`	El cliente puede corregir sus datos sin tener permiso de `UPDATE` sobre toda la tabla `cliente`. Es `SECURITY DEFINER` y solo `rol_cliente` puede ejecutarla. La cédula no se cambia aquí porque es la llave de sus ventas

9. Cuentas, roles y seguridad por filas (RLS)

Cada usuario de la tienda es una fila de `cliente` con `rol` `admin` o `cliente`. La aplicación se conecta a la base con un solo usuario, pero antes de cada consulta ejecuta `SET ROLE` con el rol de PostgreSQL que corresponde a quien usa la página. Desde ese momento es la base, y no el PHP, la que decide qué puede ver y cambiar.

Rol de PostgreSQL	Quién lo usa	Qué puede hacer
`rol_visitante`	Quien todavía no inició sesión (login y registro)	Leer `ciudad`; leer de `cliente` solo cédula, correo, hash y rol (para revisar el login); insertar un cliente nuevo sin poder elegir su rol
`rol_cliente`	Un cliente con sesión iniciada (la tienda)	Leer `articulo`, `categoria`, `ciudad` y `articulo_imagen`; leer sus datos de `cliente` (sin el hash ni el rol); leer e insertar en `venta`, `detalle_venta` y `forma_pago`, pero solo las suyas (RLS); ejecutar `fn_actualizar_mis_datos`
`rol_admin`	El administrador (el panel)	Todo sobre las 8 tablas y sus secuencias

Los tres roles son `NOLOGIN`: nadie puede conectarse directamente con ellos, solo usarlos con `SET ROLE` desde la conexión de la aplicación. En una base local, el usuario de la aplicación `app_ventas` es `NOINHERIT`: no recibe automáticamente los permisos de los roles, tiene que activarlos con `SET ROLE`.

Seguridad por filas (Row Level Security)

`venta`, `detalle_venta` y `forma_pago` tienen RLS activado: además de tener permiso sobre la tabla, cada fila tiene que cumplir una política.

Política	Tabla	Rol	Regla
pol_venta_admin, pol_detalle_venta_admin, pol_forma_pago_admin	las tres	`rol_admin`	Todas las filas (`USING (true)`)
pol_venta_cliente	venta	`rol_cliente`	Solo las ventas cuyo `id_cliente` es la cédula en sesión
pol_detalle_venta_cliente	detalle_venta	`rol_cliente`	Solo las líneas de una venta suya
pol_forma_pago_cliente	forma_pago	`rol_cliente`	Solo los pagos de una venta suya

La cédula en sesión la guarda la aplicación en la conexión con `set_config('app.id_cliente', …)`, y las políticas la leen con `current_setting('app.id_cliente', true)`. Con esto, aunque la página tuviera un error, un cliente no puede ver ni registrar compras o pagos a nombre de otro: la base devuelve sus filas y nada más.

Las contraseñas se guardan con `password_hash` de PHP (bcrypt). Los límites de intentos de login (5 intentos fallidos bloquean el correo 5 minutos) y el token CSRF de los formularios los aplica la aplicación, no la base.

10. Normalización (3FN): por qué el total no se guarda

Cada dato vive una sola vez y en la tabla a la que pertenece. Los artículos de una venta van como filas en `detalle_venta` (1FN). La cantidad depende de la llave completa venta + artículo (2FN). El departamento depende de la ciudad y no del cliente (3FN). Por eso `venta` no tiene columna total: si se guardara aparte podría quedar distinto de sus detalles. Se calcula al consultar:

```
SELECT v.id_venta, SUM(dv.cantidad * a.precio) AS total
FROM venta v
JOIN detalle_venta dv ON dv.id_venta = v.id_venta
JOIN articulo a ON a.id_articulo = dv.id_articulo
GROUP BY v.id_venta;
```

La foto va en su propia tabla, `articulo_imagen`, y no como columnas de `articulo`, porque es opcional: así `articulo` no se llena de columnas vacías y la foto depende solo de la llave del artículo. Las columnas de la cuenta en `cliente` (`password_hash` y `rol`) describen a la misma persona y dependen solo de su cédula, así que la tabla sigue en 3FN.

11. Datos con los que se llenó la base

`ventas_internet.sql` carga 24 filas de prueba en las 7 tablas principales, para poder hacer consultas desde el primer momento. Las categorías son 1 Computadores, 2 Celulares y 3 Accesorios. Todas las ciudades tienen país `'Colombia'`. Estos clientes de prueba no tienen contraseña, así que no pueden iniciar sesión en la tienda.

Ciudades

id	Nombre	Departamento	Código postal
1	Villavicencio	Meta	500001
2	Bogotá	Cundinamarca	110111
3	Medellín	Antioquia	050001

Clientes

Cédula	Nombre	Email	Teléfono	Dirección	Ciudad
1012345678	Laura Pérez	laura.perez@mail.com	3101234567	Cra 33 # 15-20	1
1023456789	Carlos Gómez	carlos.gomez@mail.com	3209876543	Calle 80 # 10-45	2
1034567890	Ana Torres	ana.torres@mail.com	3005558899	Av 70 # 45-12	3

Artículos

id	Nombre	Precio (COP)	Stock (cargado → tras las ventas)	Categoría
1	Portátil Lenovo IdeaPad 3	2.350.000	10 → 9	1 Computadores
2	Samsung Galaxy A55	1.650.000	15 → 14	2 Celulares
3	Mouse Logitech M170	45.000	50 → 47	3 Accesorios
4	Audífonos JBL Tune 520BT	199.000	20 → 19	3 Accesorios

Ventas y su detalle

3 filas en `venta` y 5 en `detalle_venta`; cada pedido se envía a la dirección del cliente.

Venta	Fecha	Cliente	Estado	Artículos (cantidad)	Total (COP)
1	2026-10-01	Laura Pérez	Pagada	Portátil Lenovo (1), Mouse Logitech (2)	2.440.000
2	2026-10-05	Carlos Gómez	Enviada	Samsung Galaxy A55 (1), Audífonos JBL (1)	1.849.000
3	2026-10-10	Ana Torres	Pendiente	Mouse Logitech (1)	45.000

Pagos

id	Tipo	Monto (COP)	Fecha de pago	Estado	Venta
1	PSE	2.440.000	2026-10-01	Aprobado	1
2	Tarjeta Visa	1.849.000	2026-10-06	Aprobado	2
3	Efecty	45.000	2026-10-10	Pendiente	3

Datos actuales en Railway

La base publicada en Railway tiene, además, los datos registrados desde la página. Al 8 de noviembre de 2026 tenía 32 ciudades, 38 artículos y 21 clientes, además de las ventas y pagos hechos desde la tienda. Las fotos de los artículos las sube el administrador desde la página y quedan en `articulo_imagen`.

12. Backup y conclusión

Para sacar un backup completo de la base se usa `pg_dump proyecto > backup-proyecto.sql`. En Railway también se puede crear uno desde la pestaña Backups del servicio Postgres. Para crear la base desde cero se ejecutan los tres scripts en el orden indicado en la introducción.

Conclusión: la base guarda cada dato una sola vez (3FN), protege el historial con llaves foráneas, valida los datos con restricciones `CHECK` y `UNIQUE`, y sus disparadores controlan el inventario y las fechas de pago. Con los roles de PostgreSQL y la seguridad por filas, cada usuario solo puede hacer lo que le corresponde: un visitante solo puede registrarse, un cliente solo ve y crea sus propias compras y el administrador maneja todo. Así, ninguna aplicación puede guardar precios negativos, vender sin stock, registrar un pago anterior a la venta, guardar estados inválidos ni mostrarle a un cliente las compras de otro.